

Petite balade en métro et en tramway

Comment les structures de données peuvent nous aider à voyager plus efficacement ?

Un Apprentissage Par Problème (APP) destiné aux étudiants
du module Algorithmique avancée 3



Etienne SCHMITZ & Henri Hagbe

Éléments de contexte

Les réseaux de transports en commun des grandes métropoles françaises sont devenus des infrastructures essentielles du quotidien. À Paris, le métro transporte plus de 4 millions de voyageurs par jour ; à Lyon, Lille et Bordeaux, les réseaux de métro et de tramway constituent l'épine dorsale de la mobilité urbaine.

Pour les usagers, planifier un trajet d'un point A à un point B n'est pas toujours simple : correspondances entre lignes, temps d'attente, choix entre un itinéraire avec moins d'arrêts ou un itinéraire plus rapide... Des applications comme celles de la RATP ou de Citymapper répondent à ce besoin, mais reposent sur des requêtes vers des serveurs distants et des bases de données propriétaires.

ESME Sudria, grande école d'ingénieurs et actrice engagée de la transition énergétique, dispose de campus dans plusieurs de ces métropoles : Paris, Bordeaux, Lille et Lyon. Dans le cadre de son plan Eco'Prepa, l'école souhaite proposer à ses nouveaux étudiants un calculateur d'itinéraires basé exclusivement sur les transports en commun, intégré au bouquet d'applications de son système d'information scolaire.

Le cahier des charges impose une contrainte forte : l'application doit fonctionner entièrement hors ligne, sans appel à des API externes ni à des outils de génération de code automatique. Les données des réseaux de transport seront fournies sous forme de fichiers structurés, et le moteur de calcul devra être suffisamment générique pour traiter n'importe quel réseau — l'ajout d'une nouvelle ville ne doit nécessiter que l'ajout d'un nouveau fichier de données, sans aucune modification du code.

Vous êtes une équipe de jeunes ingénieurs et vous avez la charge du projet...





Travail demandé

Le but de ce projet est de fournir un algorithme capable de donner le meilleur itinéraire (en termes de temps de parcours) pour relier les stations des réseaux de transport des villes suivantes :

- Le métro et le tramway parisien (lignes 1-14)
- Le réseau TBM de la ville de Bordeaux (tramways A-F, Bus express G, H)
- Le métro et le tramway de la métropole lilloise (métro 1-2, tramways T et R)
- Le métro et le tramway de la ville de Lyon (métro A-D, tramways T1-T7)

Des documents indiquant les différents plans des réseaux sont donnés en annexe.

Votre mission consiste à concevoir un outil qui permet :

1. Charger des données :

- Lire les données d'un réseau à partir d'un fichier JSON contenant :
 - Lignes : dictionnaire décrivant chaque ligne (nom, couleur, liste ordonnée des stations)
 - Connexions : liste des connexions entre stations voisines (station départ, station arrivée, temps en secondes, ligne)
 - Correspondances : liste des stations de correspondance (station, lignes desservies, temps de correspondance)
- Construire un graphe pondéré à partir de ces données
- Choisir la structure de données la plus adaptée : matrice d'adjacence ou liste d'adjacence

2. Parcourir le réseau

- Implémenter au moins un algorithme de parcours de graphe :
 - BFS (parcours en largeur) : trouve le chemin avec le moins d'arrêts
 - DFS (parcours en profondeur) : explore le graphe en profondeur
- Vérifier que toutes les stations sont accessibles depuis n'importe quelle autre station
- Identifier les stations de correspondance dans le réseau

3. Trouver le meilleur itinéraire

- Implémenter l'algorithme de Dijkstra pour calculer le plus court chemin en temps de parcours
- Intégrer le temps de correspondance (120 secondes) dans le calcul
- Reconstruire et afficher l'itinéraire de manière lisible :
 - Station de montée et ligne
 - Stations traversées

- Correspondances effectuées
- Station de descente et ligne
- Temps total de parcours

4. Généraliser à plusieurs villes

- Proposer à l'utilisateur de choisir une ville parmi celles disponibles
- Charger automatiquement les données du réseau correspondant
- Permettre de choisir une station de départ et une station d'arrivée
- Prouver la généricité du code : l'ajout d'une nouvelle ville ne nécessite aucune modification du code, seulement l'ajout d'un fichier JSON

5. Interface utilisateur (console)

- Créer un menu interactif en console permettant de :
 - Sélectionner la ville
 - Saisir les stations de départ et d'arrivée (avec gestion des erreurs de saisie)
 - Afficher l'itinéraire de manière claire et formatée
 - Relancer un calcul ou quitter
- (Optionnel) Créer une interface graphique avec PyQt5

6. Optionnel : Visualisation du graphe

- Intégrer une visualisation du réseau à l'aide de la bibliothèque *networkx* :
- Afficher le graphe du réseau avec les stations et les connexions
- Mettre en évidence l'itinéraire calculé sur le graphe
- Différencier les lignes par couleur

7. Optionnel : Gestion des perturbations

- Permettre à l'utilisateur de fermer une station (travaux, incident) : le calculateur doit recalculer un itinéraire alternatif en excluant cette station du graphe
- Permettre de fermer un tronçon entre deux stations consécutives (ligne interrompue entre deux arrêts)
- Permettre de fermer une ligne entière (grève, maintenance prolongée)
- Afficher un message d'avertissement lorsque l'itinéraire initialement demandée est impacté par une perturbation
- Comparer le temps de trajet normal vs le temps avec perturbation pour quantifier l'impact sur le voyageur



Pour simplifier le problème, nous allons faire les restrictions suivantes :

- Seules les stations énoncées dans les fichiers de données fournis sont prises en compte
- Toutes les rames sont supposées se déplacer à la même vitesse
- Les seules correspondances possibles sont celles d'une même station sur deux lignes différentes (pas de correspondance à pied du type La Chapelle — Gare du Nord)
- Toutes les correspondances prennent le même temps : 120 secondes (même si en réalité certaines correspondances demandent plus de temps)
- Les données des réseaux sont fournies — vous n'avez pas besoin de les extraire des plans

Exemples de sortie attendue

On suppose que l'utilisatrice a choisi la ville de Paris et qu'elle souhaite se rendre de la station Argentine (sur la ligne 1) à la station Monceau (sur la ligne 2) :

Monter station Argentine, ligne 1
Continuer station Charles de Gaulle - Étoile
Correspondance station Charles de Gaulle - Étoile, prendre la ligne 2
Continuer station Ternes
Continuer station Courcelles
Descendre station Monceau, ligne 2

Temps total estimé : 9 minutes 30 secondes

Toute amélioration des interactions utilisateur et ajouts de fonctionnalités utiles sont en bonus et appréciées.

Les ressources pour traiter la situation-problème

- Le cours 3 et le TD 3 « Les graphes ».
- Le cours de l'ensemble du semestre 1 du module « Algorithmique avancée 2 »
- Optionnel : Le cours d'« Interface Homme-Machine » (cours du 2nd semestre)
- Le jeu de données disponible en annexe (sous Moodle)



Livrables attendus

- Le projet respectant les exigences ci-dessus (sous le format au choix : script Python, Jupyter Notebook, ...).
- Un rapport synthétique expliquant :
 - La structure du projet.
 - Les choix techniques : structure de donnée, algorithmes utilisés, ...
 - Les difficultés rencontrées et leur résolution.

Une attention particulière sera accordée au respect des bonnes pratiques de programmation.

Objectifs d'apprentissage de l'APP (AAV)

- Représenter un réseau de transport sous forme de graphe
- Choisir la bonne structure de données (matrice d'adjacence, liste d'adjacence)
- Parcourir un graphe avec les algorithmes BFS et DFS
- Trouver le plus court chemin avec l'algorithme de Dijkstra
- Concevoir un code générique applicable à plusieurs jeux de données
- Collaborer efficacement en équipe pour structurer un projet et répartir les tâches
- Produire un code clair, documenté et modulaire
- Présenter les résultats de manière synthétique